

Future Work & Enhancements

What we would build given more time

Self-Correction

Better UI

Documentation

Demo Video

The Drone Security System was built under tight time constraints. This document captures the enhancements that would have been prioritised given more time -- not as wish-list items, but as concrete, well-reasoned extensions that follow directly from the current architecture and address real gaps in the system's operational readiness.

1

Alert Channels

Email, SMS, Slack, Teams, push notifications

2

Self-Correct Agent

Feedback loop, auto rule proposals, calibration

3

Enhanced UI

React frontend, zone map, multi-camera grid

4

Docs & DX

API reference, runbook, Docker, CI tests

[Demo](#)
[Video](#)

Real footage, edited
scenes, scripted
scenario

1. Multi-Channel Alert Delivery

[ALERTS] If we had no time constraints...

The current system stores alerts in SQLite and surfaces them in the Streamlit dashboard. In a production deployment, a security operator cannot be expected to watch a browser tab continuously. Alerts need to reach the operator wherever they are, through redundant channels.

Email (SMTP / SendGrid)

Each HIGH severity alert would trigger an email to a configurable recipient list. The email would include the alert message, timestamp, zone, severity, and an attached JPEG of the evidence frame. Implementation: a thin `AlertDispatcher` class wrapping `smtplib` or the SendGrid Python SDK, called from the alert node after `sqlite.store_alert()`. Rate-limiting would mirror the existing 5-minute cooldown to prevent inbox flooding.

SMS (Twilio)

A one-line SMS summary for critical alerts (battery critical, perimeter breach) delivered via the Twilio REST API. SMS is the highest-reliability fallback channel when internet connectivity is degraded. The message body would be: '[HIGH] perimeter_breach at main_gate 14:23 -- person detected near fence. Review dashboard.' Configurable phone numbers stored in `.env`.

Slack / Microsoft Teams webhook

A formatted Slack Block Kit message posted to a security channel. This is the most natural channel for a team of operators -- alerts appear in chronological order alongside normal team communications, can be threaded for discussion, and Slack's mobile app provides push notifications reliably. The evidence frame thumbnail would be attached as an image block. Teams support would use the Incoming Webhook connector with an Adaptive Card.

Push notifications (Firebase Cloud Messaging)

For a mobile companion app scenario, FCM would deliver a push notification to the operator's phone with a tap-to-open deep link into the Timeline tab filtered to the relevant event. This requires a small React Native or Flutter shell app, but the backend integration is straightforward: one FCM API call per alert in the dispatcher.

Dispatcher architecture

All channels would be managed by a single `AlertDispatcher` class configured via `rules.yaml` -- each rule could specify which channels to use (e.g., `battery_critical: [email, sms, slack]`; `loitering: [slack]`). Channel failures would be caught and logged without blocking the pipeline. A delivery receipt table in SQLite would track which channels confirmed delivery, enabling retry logic for failed sends.

2. Self-Correcting Rule Agent

[AGENT] If we had no time constraints...

The current rule engine is static -- rules are written once by the operator and remain fixed until manually edited. In practice, every deployment site has different lighting conditions, traffic patterns, and threat profiles that cause rules to generate false positives or miss real events. A self-correcting agent would observe the system's own performance and propose rule adjustments.

False positive feedback loop

When an operator marks an alert as a false positive in the Timeline tab (currently only 'ack' is implemented), that feedback would be recorded with the associated rule name, zone, time-of-day, and detected object classes. After accumulating N false positives for a rule, a correction agent would wake up, retrieve the flagged frames via HybridRetriever, and use an LLM to analyse the pattern: 'This loitering rule fires on maintenance staff entering the west gate every morning between 07:00 and 08:00. Suggest adding an after_hours condition or a zone exclusion.'

Missed event detection

Operators who manually acknowledge a situation in the real world but find no corresponding alert in the system would flag it via a 'missed event' button. The agent would retrieve frames from that time window, analyse what was visible, and propose a new rule or a threshold adjustment to catch it in future. This closes the feedback loop in both directions.

Automated rule proposal

The agent would output proposed YAML rule diffs -- not automatically applied, but presented to the operator in a 'Suggested Rule Changes' panel for review and one-click approval. This keeps a human in the loop for all policy changes while dramatically reducing the manual effort of rule tuning. The RuleEngine.reload() hot-reload mechanism is already in place to apply approved changes without a restart.

Threshold auto-calibration

Numeric thresholds (loitering duration, crowd size, vehicle count) could be calibrated per zone based on baseline activity patterns. A zone with naturally high footfall (a public entrance) would get a higher crowd threshold than a restricted server room. The calibration agent would run weekly, analyse the prior week's detection distribution, and propose threshold updates. SQLite already stores all the data needed for this -- no new storage infrastructure required.

3. Enhanced User Interface

[UI] If we had no time constraints...

The current Streamlit dashboard is functional but uses Streamlit's page model, which re-runs the full script on every interaction and has limited real-time capability. A production security interface would need a more responsive, purpose-built frontend.

React / Next.js frontend with WebSocket live feed

A dedicated React frontend would connect to a FastAPI backend via WebSockets. The backend would push frame results, telemetry updates, and alerts as JSON events. The frontend would render the annotated video using a canvas element, updating at the full processed FPS without the full-page rerun overhead of Streamlit. This would make the live feed feel genuinely real-time rather than polling-based.

Multi-camera grid view

A 2x2 or 3x3 camera grid showing simultaneous feeds from multiple drone sources or fixed cameras. Each tile would show a thumbnail, the active detections count, and a red border if an alert is active. Clicking a tile would expand it to the full single-camera view. The StreamProcessor architecture already supports multiple independent instances -- the frontend grid would be the main missing piece.

Interactive zone map

A top-down site map (SVG or Leaflet.js) overlaid with the defined security zones as coloured polygons. Live detection counts and recent alert counts per zone would be shown as badges on the map. Clicking a zone would filter the Timeline and Ask tabs to that zone automatically. GPS coordinates from the telemetry stream would show the drone's current position as a moving icon.

Alert acknowledgement workflow

The current Ack button is a single click with no follow-up. A production workflow would require the operator to select an outcome (false positive, investigated, escalated, resolved) and optionally add a note. This audit trail feeds directly into the self-correcting rule agent described above and provides a compliance record for security audits.

Mobile-responsive operator view

A simplified mobile layout showing only the active alert feed, a one-tap acknowledge button, and the most recent annotated frame. Designed to be used on a phone when the operator is on-site and away from a desktop. Progressive Web App packaging would allow it to be installed and receive push notifications via the FCM channel described above.

4. Documentation & Developer Experience

[DOCS] If we had no time constraints...

The current documentation covers architecture, design decisions, testing, and AI tooling. With more time, the focus would shift to operational documentation that enables another developer or operator to set up, configure, and extend the system without needing to read the source code.

Interactive API reference (Swagger / MkDocs)

A FastAPI backend would expose all pipeline operations as REST endpoints (ingest video, query footage, acknowledge alert, reload rules) with auto-generated Swagger documentation. MkDocs Material would host the full developer reference including auto-generated docstring pages for every public class and function, cross-linked with the architecture diagrams.

Operator runbook

A step-by-step guide written for a non-technical security operator: how to start the system, what each alert severity means, how to acknowledge and escalate events, how to ask questions in the Ask tab, and what to do when the drone battery warning fires. Written in plain language with annotated screenshots, not code snippets.

Rule authoring guide

A dedicated guide for security managers who need to write or modify rules in rules.yaml. Would cover every supported condition type with examples (time_range, class_count, duration_above, battery_below, caption_contains), the needs_l1m escalation flag, severity levels, message template variables, and common patterns (after-hours rules, zone-specific rules, telemetry rules). Would include a rule validation script that checks the YAML for schema errors before deployment.

Docker Compose one-command setup

A docker-compose.yml that brings up all services -- the pipeline worker, the Streamlit dashboard, the alert dispatcher, and a ChromaDB container -- with a single 'docker compose up'. Currently requires manual environment setup and assumes a local Python environment. Containerisation would make deployment reproducible and portable, and would enable cloud deployment on AWS ECS or Azure Container Apps with minimal additional configuration.

End-to-end scenario tests with synthetic video

Programmatically generated synthetic video clips (a moving bounding box simulating a person loitering, a sequence of vehicle entries) would enable fully deterministic end-to-end tests that verify the complete pipeline from raw video input to stored alert output. These tests would be included in CI and would catch regressions in the rule engine, LangGraph routing, and storage layer in a single automated run.

5. High-Quality Demo Video

[DEMO] If we had no time constraints...

The current demo video is a screen recording with voiceover covering the core functionality. A polished submission-quality demo would communicate the system's value more effectively and be appropriate for a broader audience including non-technical stakeholders.

Edited multi-scene structure

Rather than a single continuous screen recording, the video would be edited into clearly labelled scenes with title cards: Problem Statement, Architecture Overview, Live Detection Demo, Alert Escalation, Chatbot Query, Telemetry Monitoring, and Summary. Scene transitions would be clean cuts with a brief title overlay. Total runtime would be kept under eight minutes with tighter pacing than an unedited recording allows.

Real drone footage

The demo would use real aerial footage captured by a consumer drone over a property, rather than a stock video file. This would demonstrate that the YOLOv8 detection pipeline performs correctly on real drone perspective footage (top-down angle, motion blur, varying altitude) and would make the security use case immediately intuitive to viewers.

Annotated screen captures

Key moments in the video would be paused and annotated with callout arrows pointing to the relevant UI elements being discussed -- for example, an arrow pointing to the track ID badge on a bounding box while explaining loitering detection, or a callout highlighting the telemetry battery percentage turning red as it drops below the threshold.

Scripted alert scenario

Rather than waiting for a rule to fire organically during the demo, a scripted scenario video would be prepared: a short clip where a person enters a forbidden zone, the YOLO tracker assigns a track ID, the loitering timer accumulates, the rule fires, the LLM judge confirms it as genuine, and the alert appears in the dashboard and pings a Slack channel -- all within 90 seconds. This gives a clean, repeatable narrative arc.

Professional audio

The voiceover would be re-recorded in a quiet environment using a dedicated microphone with a pop filter, rather than a laptop microphone. Light post-processing (noise reduction, normalisation) would be applied. A short background music bed at low volume during non-narrated transitions would improve perceived production quality significantly.